把 LLM 锁进 schema 里逐层建图 Constructing Dynamic Master Logic Models as Knowledge Graphs for Complex System Diagnostics Using Retrieval-Augmented Large Language Models

本篇范围说明：这是一篇 36 页的期刊投稿式论文（马里兰大学风险与可靠性中心 Saman Marandi、Mohammad Modarres，与台湾新竹 DML Inc. 的 Yu-Shu Hu）。下面「全文翻译」按原文章节推进，覆盖摘要、§1 引言、§2 背景（三小节）、§3 方法总览、§4 框架（构建/评测/交互三大块）、§5 案例、§6 结果、§7 讨论与局限与未来工作、§8 结论。两段 Cypher 生成与评测的伪代码（Algorithm 1、2）只说它们干什么，不逐行译；参考文献列表与附录 A 的逐次运行明细不译。

核心内容

Q：DML 是什么？它和故障树有什么本质区别？ A：DML 的全称是 Dynamic Master Logic，动态主逻辑，是一种**功能建模**框架，由本文作者之一 Hu 与 Modarres 提出。它把系统画成一个从上到下的层次：系统目标（Goal）→ 功能（Function）→ 子功能（Subfunction）→ 部件（Component）→ 成功条件（SuccessCondition），层与层之间用 AND/OR 逻辑门连起来。

和故障树分析（Fault Tree Analysis，FTA）、事件树分析（Event Tree Analysis，ETA）的区别在于**问的问题不一样**。故障树是事件驱动的：你得先把可能的故障事件列全，再顺着预设的因果链往下追。系统一复杂，这个清单就永远列不全，没预料到的失效序列压根诊断不了。功能建模问的是「这个系统本来要达成什么、这些目标怎么分解成功能、这些功能靠哪些部件实现」，故障是通过**偏离预期功能**被检测出来的（比如流量丢失、信息丢失），不需要事先枚举它。作者的说法是：工程系统本来就是按目标和功能架构来记录和构想的，故障树是这份功能知识的**下游抽象**，拿下游当因果推理的基础，天生就不完整。

Q：这篇论文到底做了什么新东西？ A：DML 本身不新，新的是**自动构建**。以前建一个复杂系统的 DML，靠专家读技术文档、手工识别目标和功能、手工拼层次，作者说通常要几个月。这篇论文用检索增强生成（Retrieval-Augmented Generation，RAG）加大语言模型，从工程文档里自动把这个层次抽出来，存成知识图谱（论文叫 KG-DML），并且配了一套三层的评测方法。

作者自己明确说：新意不在于单独使用 RAG、LLM、schema 约束抽取或图数据库这些现成技术，而在于**把它们集成成一个能从工程文档自动建 DML 的连贯框架**，加上一个可执行的图表示和一套讲道理的评测方法。

Q：它怎么防止 LLM 乱编？ A：这是全文最值得学的一点。作者不是靠提示词让模型「小心一点」，而是**削减模型的自由度**，四道约束叠在一起：

1. **预定义 schema**：给 LLM 一个从 DML 形式化推导出来的结构化模板，规定了有哪些节点类型、哪些关系、层次怎么排。模型不能自己发明节点类型。
2. **有界范围**：抽取任务限定在「正在建模的这个系统」之内，不是开放域。
3. **逐层构建 + 父节点条件化**：不一次性推整个模型。每一层单独建，检索查询由两样东西拼出来——这一层的语义定义，加上上一层已经确定的父节点。也就是说，每次抽取的搜索范围都被已经建好的那部分**框住**了。
4. **输出受 JSON schema 约束 + 事后校验**：模型输出必须符合预定义的 JSON 格式，生成完还要检查结构正确性、标识符有效性、这一层特有的约束；不过关的记日志，那一层标成低置信度。

作者的原话是，这些设计合起来把 LLM 的角色**从无约束生成变成受控框架内的结构化解释**。

Q：知识图谱在这里承担什么角色？和本体（ontology）是什么关系？ A：这一段是全文对做语义建模的人最有价值的地方，作者说得很硬：

KG 只是实现和存储机制，不定义功能结构。 DML 定义了功能结构（目标、功能、依赖、成功逻辑），KG 只负责把这些东西编码成机器可读的形式，支持存储、遍历和推理，它本身不定义任何功能语义。用 KG 是因为图结构灵活、能自然容纳层次分解和逻辑构造，不是因为图有什么魔力。

DML 和传统本体的区别：传统本体建的是静态的领域词汇、分类层级和语义实体，底子是灵活的关系图——概念、属性、语义关系可以往任意方向连，而它的推理主要是为**类成员推断**（class-membership inference）设计的，不是为功能系统建模设计的。DML 反过来，强制一个严格的**目的-手段层次**（means-end hierarchy），分解只能从系统级目标往下走到子系统和部件功能，而且带**显式的成功逻辑**，规定了要成功实现某个功能需要满足哪些运行条件。作者说，正是「严格层次分解」加「显式成功逻辑」这个组合，让 DML 能支持关于系统行为、功能依赖和失效传播的智能推理，而通用本体框架**不是为这件事设计的**。

Q：评测怎么做？为什么要造一个新分数？ A：因为诊断推理是靠在图上做确定性遍历和布尔传播完成的，所以**图的结构质量直接决定诊断结论的质量**。少一个节点、连错一条关系、依赖路径不完整，错误都会顺着模型传下去，给出错的诊断。

评测分三层：

- **节点层**：每层分开算 precision（识别出来的节点里多少是对的）、recall（参考模型里的节点有多少被找回来了），另报 **F2 分数**。用 F2 而不是 F1，是因为它更偏重 recall——**在 DML 里漏掉一个元素通常比多出一个元素更有害**。
- **关系与逻辑层**：关系错误分成漏连、连错、多连；逻辑门单独评（父节点到子节点之间的 AND/OR 是不是和参考模型一致）。门评测和节点、连接分开做，因为**门错了会改变元素对上层功能的贡献方式，即使节点全对**。
- **整体完整度**：一个 IntegrityScore = 100 × 各层 F2 的加权平均 × exp(−结构惩罚 ÷ 参考模型规模 ÷ 惩罚尺度)。前半截管元素级准确性，后半截是指数惩罚项，收漏节点、漏关系、门不匹配、孤立节点。用指数形式是因为结构错误对功能推理的影响是非线性的——零星几个错扣得温和，累积起来扣得陡。分母做了规模归一化，所以不同大小的系统之间可比。

Q：结果怎么样？作者对自己的成果有多少保留？ A：案例是一座已退役沸水堆（Millstone Point Unit 1）的低压冷却剂注入系统（LPCI），文档来自美国核管会的一份可靠性评估报告。跑了五次独立执行：

- **高层完美**：Goal、Function、Subfunction 三层的 precision、recall、F2 五次全部等于 1.0。Goal-Function 和 Function-Subfunction 这两层关系也是五次全对。
- **低层小幅掉分**：Component 层 F2 在 0.978～0.990（每次漏 1-3 个节点），SuccessCondition 层 F2 在 0.980～0.993（每次漏 2-6 个）。
- **最密的那一层最不稳**：Subfunction-Component 关系层 recall 0.951～0.978，**门准确率 0.889～0.963**。作者的解释是这一层结构密度最高、嵌套逻辑最多。
- **批大小的实测曲线**：batch=1 得 90.38 分、batch=5 得 88.98、batch=10 得 87.98、batch=15 得 86.35，单调下降。但从 5 降到 1，LLM 调用次数从约 32 次涨到约 158 次（近五倍），换来的只有 1.4 分。作者的结论是 **batch=5 是正确性与成本的平衡点**。

保留说得很足：框架是**辅助而非替代**专家建模，生成的模型应当视为「LLM 提供信息的」（LLM-informed），最终审核与验证仍是领域专家的责任，尤其是低层细节。「几个月 → 几天」这个说法，作者在局限一节里自己标明是「基于有限的实践经验，不是对照实验」。

背景补充 / 点评

几个术语的出处

GTST-MLD 是 DML 的前身，全称 Goal Tree-Success Tree and Master Logic Diagram（目标树-成功树与主逻辑图）。它的构造规则是：高层目标说明**为什么**需要某些功能和结构，低层功能说明这些目标**如何**通过硬件、软件、人的行为的交互被达成。DML 在这个基础上加了时间相关、不确定、演化的行为，所以名字里多了 Dynamic。这条演进链在论文里写得很清楚：事件驱动的 FT/ET → GTST 与 GTST-MLD 的功能层次 → DML 的动态推理 → 现在机器可读的 KG-DML，一条持续在改善「完整性、可扩展性、计算可达性」的线。

CCF 是 Common Cause Failure，共因失效——多个部件因为同一个原因同时坏掉（共用设计、放在同一个位置、同一套维护规程、同样的环境暴露）。这在核安全领域是个大题目，因为冗余设计的全部价值都建立在「两条路不会同时断」这个假设上。本文对它的处理是**只做定性提示、不进结构**：当向下遍历发现多条成功路径里出现相同或冗余的部件时，解释层会加一句「这些部件可能存在共因失效风险」，但不改布尔逻辑、不改路径生成。作者诚实地把这标为「部分处理」。

F2 分数：F-beta 家族的一员，beta=2 时 recall 的权重是 precision 的两倍。选它而不选常见的 F1（两者等权），是一个**领域判断**而不是技术偏好——在安全分析里，漏掉一条失效路径的代价远大于多标一条不存在的路径。

这篇的洞察在哪、留白在哪

洞察最深的是那段 DML 与本体的对比。 它没有停在「本体是静态词汇、DML 是功能层次」这种表层差异上，而是指出了推理目标的分歧：**本体推理是为类成员推断设计的，不是为功能推理设计的。**这句话解释了一个常见困惑——为什么把领域知识写成 OWL 本体、开了推理机，却还是答不出「这个部件坏了会影响哪个目标」。因为本体推理机能告诉你「泵是设备的子类」，但它不知道「A 泵和 B 泵是 OR 关系，坏一个还行，坏两个就完了」——那个 OR 不是分类学关系，是**成功逻辑**，得显式建出来。

第二个洞察是把 LLM 的角色明确降级。 交互阶段的分工写得很干净：LLM 只负责解释用户意图、选并调用合适的图工具；概率和逻辑推理由编码在模型结构里的东西执行。作者说这个分离「有助于保持可追溯性、限制交互过程中无依据的推理」。这不是保守，是把不可靠的部分限制在它可靠的能力上——理解自然语言这件事 LLM 确实强，多步一致推理它确实弱，那就只让它做前者。

最大的留白：没做消融，而恰恰该做消融的正是它的两个卖点。 作者给的理由是文档量远超上下文窗口和生成上限，所以「检索是框架的结构性必需，不是可以拿来做基线对照的设计选项」——这个理由对「有检索 vs 无检索」这一对是成立的。但它的贡献声明里有两条完全可以做消融、也完全没做：**逐层构建 vs 一次性构建、父节点条件化检索 vs 不条件化。**这两条才是这篇区别于普通「LLM 建 KG」的地方。作者在局限里承认了「systematic ablation studies 是未来工作」，还额外辩解说开放域 KG 文献的消融范式不直接迁移到 schema 约束的层次构建任务上——这个辩解有一半道理（确实没有「无 schema 的 DML」这种基线），但另一半不成立：层次构建的方式本身就是可以对照的。

第二处留白：IntegrityScore 不可复现。 论文说层权重「反映各 DML 层对下游推理的相对影响」、「在所有实验中保持不变」，但**没给出具体数值。**惩罚尺度 S 做了 0.4/0.5/0.6 的敏感性分析，层权重却一个数都没有。加上门不匹配惩罚系数 λ_gate 也没给值，这个 90.38 分别人复算不出来。论文自己承认「不同的加权方案或惩罚形式，即使底层结构差异相当，也可能得出不同的数值分数」——那这个分数就只能做**同一份实现内部的相对比较**，不能跨论文比。

第三处：只评结构，不评功能等价。 作者对此的自我批评比我能给的更狠，值得原样记下：结构相似不保证向上/向下传播时的逻辑行为相同；更强的检验是比较**最小割集或成功路径集**；但即便割集和路径集完全等价，**也不构成 DML 模型的完整功能验证**，因为这些指标停在结构部件层，捕捉不到「目标如何分解成功能和子功能」以及「失效如何在这个功能层次里传播」。所以专家对功能层的验证仍然是必要补充。这段话本身就是一篇小教材：**它区分了三个层次的等价——结构等价、行为等价、功能等价——并且说明为什么前两个都不足以替代第三个。**

对我的启示

一、我正在建的两套模型都有「层次」，这篇给了「逐层建 vs 一次性建」的答案。 UMU 那份知识点/技能统一模型 spec（`docs/core-business/spec/2026-09-08-knowledge-skill-统一模型-spec.md`）现在是四张表加一套映射，本质是个层次结构。prove 里的 tbox/rbox 也是。这篇的做法可以直接搬：**不要指望一次把整个模型抽出来，按层推进，每一层的检索范围由上一层已确定的节点框住。** 它的量化证据很有说服力——高三层五次运行全部 100%，掉分全在低层。这不是偶然，是因为高层的父节点少、约束强、检索范围窄。

二、batch size 那张表是「便宜 vs 正确」的标准写法，我该照着写。 我的全局规则里有一条「提方案对比时把便宜和正确分开列，如果推荐的是便宜那个必须明说」。这篇就是这么做的：batch=1 分数最高（正确），但调用涨五倍只换 1.4 分，所以推荐 batch=5（便宜），并且**把代价和收益的具体数字都摆出来**让读者自己判断。我以后写方案对比表，这个格式可以直接抄——不是「推荐 A，因为 B 太贵」，而是「A 得 88.98 分、32 次调用；B 得 90.38 分、158 次调用；我推荐 A，因为多付五倍换 1.4 分不值，但如果你的场景里结构错误的代价极高，选 B」。

三、「KG 只是存储机制不定义结构」这句话，正好是我 spec 里那个判断的另一种说法。 我把 `kp_nature`（结构性/装饰性）放在「课程 × 知识单元」关系表上而不是放在实体上，理由是「同一个案例换一门课角色会变」。这篇讲的是同一件事的另一面：**形式（用图还是用表）和语义（什么定义了什么）必须分开。**DML 定义功能结构，Neo4j 只是把它编码；我的模型定义知识单元和它的场景角色，MySQL 或者图库只是把它存下来。以后有人问「为什么不用图数据库」，答案不是「表也够用」，而是「选存储和定语义是两个决策，先把语义定清楚，存储按查询模式选」。

四、那段本体推理的批评，我要拿去检查自己的 RDFS/SHACL 选择。 我之前发现过「`rdfs:domain` 是推理规则不是约束」——写了 `rdfs:domain` 不会拦住违规数据，反而会**给违规数据补上一个类型。**这篇从另一个角度说到了同一处：本体推理是为类成员推断设计的。两个观察合起来是一条清楚的判据：**凡是我想表达「必须有」「不能超过」「至少一个」这类语义，都不能指望本体推理，要用 SHACL 这类真约束，或者像这篇一样把逻辑显式建成门节点。** 我的 `rbox.yaml` 里如果有靠 domain/range 暗示约束的地方，都要复查。

五、F2 而不是 F1 这个选择提醒我一件事：口径里的权重也是业务判断。 昨天核 UMU 的正确率口径时，发现产品文案说「先算每题正确率再平均」、代码是「所有提交合起来除」，两者对「反复重考」的引导正好相反。那个差别的本质和 F1/F2 是一类——**分子分母怎么组合，隐含了「什么错误更该被惩罚」这个判断**，而这个判断必须写出来、被人确认，不能藏在实现里。这篇把它写在正文（漏比多更有害），产品那份文案没写（谁的权重更大没说），差别就在这里。

六、它的自我批评给了我一个检查清单。 「结构等价 ≠ 行为等价 ≠ 功能等价」这个三分法，可以直接用在我做数仓迁移验证上。我前几天算过：把 `knowledge_point_resource` 拆两层加拉链之后，怎么证明新模型和老模型算出来的是同一件事？按这个三分法——**行数对得上是结构等价，抽查几个学员的正确率一致是行为等价（在观测到的样本上），而「所有出题方式一条规则」这件事是否真的成立需要功能验证**（比如构造一个「题库题在两次考试之间改过绑定」的场景，验证两条路径给出同一个答案）。光对行数就宣布迁移正确，正是这篇批评的那种「拿结构相似当行为等价」。

文中金句

> "Without an explicit structured representation, an LLM cannot reliably reason about which functions support which objectives or how a component failure cascades through the system." 没有显式的结构化表示，LLM 无法可靠地推理哪些功能支撑哪些目标、或者一个部件的失效如何在系统中级联。

> "Together, these design choices shift the role of the LLM from unconstrained generation to structured interpretation within a controlled KG construction framework." 这些设计选择合起来，把 LLM 的角色从无约束的生成，转变为在受控的知识图谱构建框架内做结构化解释。

> "It is important to distinguish between the DML model and its KG representation: while DML defines the functional structure including objectives, functions, dependencies, and success logic, the KG serves only as the implementation and storage mechanism that encodes these constructs in machine-readable form, supporting storage, traversal, and reasoning without defining the functional structure itself." 必须区分 DML 模型和它的 KG 表示：DML 定义功能结构——包括目标、功能、依赖和成功逻辑——而 KG 只是实现和存储机制，把这些构造编码成机器可读的形式，支持存储、遍历和推理，但它本身不定义功能结构。

> "An F2 score is also reported to place greater emphasis on recall, reflecting the fact that missing elements in a DML model are generally more detrimental than the inclusion of extra elements." 同时报告 F2 分数以更偏重 recall，这反映了一个事实：在 DML 模型里，漏掉元素通常比多包含元素更有害。

> "Structural similarity does not necessarily guarantee identical logical behavior during upward or downward propagation." 结构相似并不必然保证向上或向下传播时的逻辑行为相同。

全文翻译

摘要

Dynamic Master Logic (DML) provides a hierarchical framework for representing system behavior by linking functional objectives to underlying structural elements. Unlike event-based approaches such as Fault Tree Analysis, DML captures system objectives, functional dependencies, and unanticipated failure scenarios within a unified functional hierarchy. However, DML construction typically relies on expert interpretation of technical documentation, limiting scalability for complex systems.

动态主逻辑（DML）提供了一个层次化框架，通过把功能目标与底层结构元素连接起来表示系统行为。与故障树分析这类基于事件的方法不同，DML 在一个统一的功能层次内同时捕捉系统目标、功能依赖以及未预料到的失效情景。然而 DML 的构建通常依赖专家对技术文档的解读，这限制了它在复杂系统上的可扩展性。

This study presents a framework for automated construction of DML models from system descriptions and their representation as Knowledge Graphs (KG-DML), leveraging Retrieval-

Augmented Generation and Large Language Models as the enabling tools. Building upon prior work on small-scale systems, the present work introduces automated construction and evaluation of KG-DML models, extending the paradigm to substantially larger and more complex system.

本研究提出一个框架，用于从系统描述自动构建 DML 模型并把它表示为知识图谱（KG-DML），以检索增强生成（RAG）和大语言模型作为使能工具。在此前针对小规模系统的工作基础上，本文引入了 KG-DML 模型的自动构建与评测，把这一范式扩展到规模大得多、复杂得多的系统上。

The proposed approach performs model construction across the hierarchy using targeted retrieval, preserving hierarchical dependencies and explicit logical relationships. The resulting KG-DML supports diagnostic reasoning, safety assessment, and graph-based analysis through upward failure propagation and downward dependency tracing. The framework is evaluated through a multi-level validation methodology that combines layer-specific precision and recall, logical gate consistency analysis, and an aggregate integrity metric capturing both element-level accuracy and system-level structural completeness.

所提方法用有针对性的检索在整个层次上进行模型构建，保留层次依赖和显式的逻辑关系。得到的 KG-DML 通过向上的失效传播和向下的依赖追溯，支持诊断推理、安全评估和基于图的分析。该框架通过一套多层验证方法评测，这套方法结合了分层的 precision 与 recall、逻辑门一致性分析，以及一个同时反映元素级准确性和系统级结构完整性的聚合完整度指标。

Application to the Low-Pressure Coolant Injection system of a decommissioned Boiling Water Reactor demonstrates consistent reconstruction across repeated runs and highlights factors influencing model stability. The results show that automated KG-DML construction can transform technical documentation into executable functional models for diagnostic and reliability analysis.

在一座已退役沸水堆的低压冷却剂注入系统上的应用表明，重复运行之间的重建结果是一致的，并揭示了影响模型稳定性的因素。结果显示，自动化的 KG-DML 构建能把技术文档转化为可执行的功能模型，供诊断与可靠性分析使用。

1. 引言：为什么事件驱动的方法不够用

Modern engineered systems are often composed of many interacting components organized across multiple levels of abstraction. Therefore, modeling these systems accurately requires representations that can account for complex interactions and dependencies. Traditional diagnostic models such as Event Tree Analysis (ETA) and Fault Tree Analysis (FTA) rely on event-driven representations in which faults or initiating events are traced through predefined causal sequences. While effective for known failures, event-driven models require exhaustive fault-event listings, making them inherently incomplete when failure sequences are not anticipated during model development.

现代工程系统往往由许多相互作用的部件组成，这些部件横跨多个抽象层级。因此要准确建模这类系统，需要能够解释复杂交互与依赖的表示方式。传统诊断模型如事件树分析（ETA）和故障树分析（FTA）依赖事件驱动的表示，即把故障或触发事件沿预定义的因果序列往下追踪。这类模型对已知失效是有效的，但事件驱动模型要求穷举故障事件清单，因此当失效序列在建模阶段没有被预料到时，它们天生就是不完整的。

Functional modeling offers an alternative approach for complex systems. Functional models describe what the system is intended to accomplish, how objectives are decomposed into functions, and how those functions are realized through interactions among components. By focusing on functions and their dependencies rather than individual events, functional models provide a clearer representation of how component behavior contributes to system-level objectives under both normal and abnormal conditions.

功能建模为复杂系统提供了另一条路。功能模型描述的是：系统打算达成什么、目标如何被分解成功能、这些功能又如何通过部件之间的交互得以实现。由于关注的是功能及其依赖而不是单个事件，功能模型能更清楚地表示部件行为在正常和异常条件下如何贡献于系统级目标。

Dynamic Master Logic (DML), introduced by Hu and Modarres, is a functional modeling framework that represents system behavior through a hierarchy linking functional objectives to underlying structures. In practice, however, constructing DML models has largely relied on manual, expert-driven processes, where developers analyze system descriptions and specifications to interpret system behavior and construct the model. This limits scalability and restricts the use of explicit functional representations needed for reliable reasoning over complex engineering systems documented through extensive technical documentation.

动态主逻辑（DML）由 Hu 与 Modarres 提出，是一种功能建模框架，通过一个把功能目标连接到底层结构的层次来表示系统行为。但在实践中，构建 DML 模型基本上依赖人工的、由专家驱动的过程：开发者分析系统描述和规格说明，解读系统行为，然后把模型搭出来。这限制了可扩展性，也限制了显式功能表示的使用——而要对那些由大量技术文档记录的复杂工程系统做可靠推理，恰恰需要这种表示。

Recent advances in Artificial Intelligence, particularly Large Language Models (LLMs), create new opportunities to support the construction of functional models. However, LLMs also exhibit important limitations, including inconsistent reasoning, lack of persistent structure, and the potential to generate unsupported information. Without an explicit structured representation, an LLM cannot reliably reason about which functions support which objectives or how a component failure cascades through the system. Dynamic Master Logic addresses this limitation by linking objectives, functions, and dependencies into an explicit structured representation that supports traversal and enables LLM agents to reason about functional dependencies and the consequences of component failures, rather than relying solely on generalized language model knowledge.

人工智能的近期进展，特别是大语言模型（LLM），为支持功能模型的构建创造了新机会。但 LLM 也表现出一些重要局限，包括推理不一致、缺乏持久结构、以及可能生成没有依据的信息。没有显式的结构化表示，

LLM 无法可靠地推理哪些功能支撑哪些目标、或者一个部件的失效如何在系统中级联。DML 通过把目标、功能和依赖连接成一个支持遍历的显式结构化表示来解决这一局限，使 LLM agent 能够推理功能依赖和部件失效的后果，而不是仅仅依赖语言模型的泛化知识。

Since diagnostic reasoning in the proposed framework is performed through deterministic graph traversal and Boolean propagation over the KG-DML, the structural quality of the KG-DML directly determines the quality of the resulting diagnostic inferences. During interaction, the LLM agent is responsible only for interpreting user queries and invoking the appropriate graph-based tools. Errors such as missing elements, incorrect relationships, or incomplete dependency paths can therefore propagate through the model, leading to inaccurate or misleading diagnostic conclusions.

由于所提框架中的诊断推理是通过在 KG-DML 上做确定性图遍历和布尔传播完成的，KG-DML 的结构质量**直接决定**了由此得出的诊断推断的质量。在交互过程中，LLM agent 只负责解读用户查询并调用合适的基于图的工具。因此，诸如元素缺失、关系错误、依赖路径不完整这类错误会在模型中传播，导致不准确或有误导性的诊断结论。

This work makes three main contributions. First, it introduces an automated approach for constructing DML models from system documentation, leveraging retrieval-augmented LLMs as enabling tools, enabling functional decomposition and dependency identification from unstructured text. Second, it extends our previous small-scale approach to support construction for substantially larger, document-intensive systems. Third, it develops a structured, multi-level evaluation framework for assessing the constructed KG-DML. The novelty of this work lies not in the individual use of established technologies such as RAG, LLMs, schema-constrained extraction, or graph databases, but in their integration into a coherent framework.

本工作有三项主要贡献。第一，提出一种从系统文档自动构建 DML 模型的方法，以检索增强的 LLM 作为使能工具，实现从非结构化文本进行功能分解与依赖识别。第二，把我们此前的小规模方法扩展到规模大得多、文档密集的系统。第三，开发了一套结构化的多层评测框架来评估构建出的 KG-DML。**本工作的新意不在于单独使用 RAG、LLM、schema 约束抽取或图数据库这些成熟技术，而在于把它们集成成一个连贯的框架。**

2.1 功能建模、DML，以及它和本体的分歧

Traditional diagnostic methods, such as ETA, FTA, and rule-based expert systems, rely on modeling discrete events and predefined failure sequences. While effective for known fault-event combinations, these event-driven approaches are often incomplete for diagnostic purposes, as unanticipated sequences cannot be diagnosed, and complexity makes failure tracing impractical. In contrast, functional models represent a paradigm shift in reliability analysis. This shift is critical as system knowledge is inherently functional. Engineering systems are traditionally documented and conceptualized through their operational objectives, functional architectures, operating and mechanistic processes, rather than their associated failure modes

and events. Event-based methods such as FTA and ETA represent a downstream abstraction of this functional knowledge and are often incomplete as a basis for causal reasoning.

传统诊断方法如 ETA、FTA 和基于规则的专家系统，依赖对离散事件和预定义失效序列的建模。它们对已知的「故障-事件」组合有效，但用于诊断时往往不完整：未预料到的序列诊断不了，而复杂度又让失效追溯变得不切实际。相比之下，功能模型代表了可靠性分析中的一次范式转变。这一转变很关键，因为**系统知识本质上是功能性的**。工程系统在传统上是通过其运行目标、功能架构、运行与机理过程来记录和构想的，而不是通过与之关联的失效模式和事件。FTA、ETA 这类基于事件的方法是这份功能知识的**下游抽象**，作为因果推理的基础往往不完整。

Building on these principles, DML provides a hierarchical framework for representing system objectives and goals, and the functions required to achieve them. Further, the model provides structural elements such as hardware operations, software executions, and human actions that realize those functions. The model is historically developed by applying the Goal Tree-Success Tree and Master Logic Diagram (GTST-MLD) construction rules. In GTST-MLD, higher-level goals define why functions and structures are necessary, while lower-level functions describe how those goals and functions are achieved through interactions among hardware, software, and human elements. Logical relationships are represented explicitly through logic gate structures that encode functional dependencies.

在这些原则基础上，DML 提供了一个层次化框架，用于表示系统目标与目的，以及达成它们所需的功能。此外，模型还提供实现这些功能的结构元素，如硬件操作、软件执行和人的行为。这一模型在历史上是通过应用「目标树-成功树与主逻辑图」（GTST-MLD）的构造规则发展起来的。在 GTST-MLD 中，**高层目标定义为什么需要某些功能和结构，低层功能描述这些目标和功能如何通过硬件、软件与人的元素之间的交互被达成**。逻辑关系通过编码功能依赖的逻辑门结构显式表示。

Within the DML framework, two fundamental causal relationships can be inferred: the propagation of failures to their ultimate system-level consequences, and the identification of functional or structural dependencies necessary for successful subsystem operation.

在 DML 框架内可以推断出两种基本因果关系：失效向其最终系统级后果的传播，以及子系统成功运行所必需的功能或结构依赖的识别。

While the hierarchical architecture exhibits structural similarities to an ontology, the DML framework fundamentally diverges from conventional ontology-based knowledge representation. Traditional ontologies model static domain vocabulary, taxonomic classifications, and semantic entities. They are built on flexible relational graph structures in which concepts, properties, and semantic relationships may connect entities in any direction, with reasoning designed primarily for class-membership inference rather than functional system modeling. Conversely, the DML framework enforces a rigorous means-end hierarchy in which the decomposition flows strictly from the system-level objective down to subsystem and component functions and incorporates explicit logic that governs the operational conditions required to successfully realize the

functions and attain the end objective. It is this combination of strict hierarchical decomposition and explicit success logic that makes DML suitable for intelligent reasoning about system behavior, functional dependencies, and failure propagation in ways that general-purpose ontology frameworks are not designed to support.

虽然这种层次架构在结构上与本体（ontology）有相似之处，DML 框架与传统的基于本体的知识表示**存在根本分歧**。传统本体建模的是静态的领域词汇、分类学分类和语义实体。它们建立在灵活的关系图结构之上，其中概念、属性和语义关系可以在任意方向上连接实体，而**其推理主要是为类成员推断（class-membership inference）设计的，不是为功能系统建模设计的**。相反，DML 框架强制一个严格的目的-手段层次，其分解严格地从系统级目标向下流到子系统与部件功能，并纳入显式逻辑来规定成功实现这些功能、达成最终目标所需的运行条件。正是「严格的层次分解」与「显式的成功逻辑」这一组合，让 DML 适合对系统行为、功能依赖和失效传播做智能推理，**而通用本体框架并非为支持这些而设计**。

In the present work, a KG is used as the underlying representation because its flexible graph structure can naturally accommodate the hierarchical decomposition, dependency relationships, and logical constructs required by the DML formalism. It is important to distinguish between the DML model and its KG representation: while DML defines the functional structure including objectives, functions, dependencies, and success logic, the KG serves only as the implementation and storage mechanism that encodes these constructs in machine-readable form, supporting storage, traversal, and reasoning without defining the functional structure itself.

在本工作中，采用 KG 作为底层表示，是因为它灵活的图结构能自然容纳 DML 形式化所要求的层次分解、依赖关系和逻辑构造。**必须区分 DML 模型和它的 KG 表示**：DML 定义功能结构，包括目标、功能、依赖和成功逻辑；而 KG 只是实现和存储机制，把这些构造编码成机器可读的形式，支持存储、遍历和推理，**但它本身不定义功能结构**。

2.2 LLM 的长处与局限

Hallucination may occur when the available information is incomplete or when problem formulations are ambiguous, leading to the generation of outputs that are plausible but incorrect. To mitigate this issue, RAG methods incorporate external knowledge sources during inference, grounding model outputs in retrieved evidence and significantly reducing hallucination in knowledge-intensive tasks. In addition, performance can degrade for tasks that require consistent multi-step reasoning or explicit verification. This limitation has motivated the coupling of LLM-based agents with external tools, including prewritten code or functions, analytical solvers, databases, and rule-based systems. Another challenge is output variability across runs, particularly for complex tasks. This work does not aim to introduce new techniques to overcome these limitations, but rather it applies established mitigation strategies reported in the literature.

当可用信息不完整、或问题表述含糊时，可能出现幻觉，产生看似合理但实际错误的输出。为缓解这一问题，RAG 方法在推理时引入外部知识源，把模型输出锚定在检索到的证据上，显著降低知识密集型任务中的幻觉。此外，对需要一致的多步推理或显式验证的任务，性能会下降。这一局限促使人们把基于 LLM 的 agent 与外部工具耦合，包括预先写好的代码或函数、分析求解器、数据库和基于规则的系统。另一个挑战是**多次运行之间的输出变异性**，在复杂任务上尤其明显。本工作不打算提出克服这些局限的新技术，而是应用文献中已有的缓解策略。

2.3 用 LLM 做故障诊断：已有工作缺了哪一块

Although these studies demonstrate the value of combining LLMs, KGs, and retrieval mechanisms, the resulting graph structures are generally used to organize domain knowledge, support information retrieval, or enhance diagnostic reasoning. To the best of our knowledge, limited attention has been given to the automated construction of executable functional models that preserve hierarchical functional dependencies and logical relationships required for diagnostic propagation. This distinction is particularly important for DML-based representations, where the model's structure directly determines the validity of diagnostic reasoning.

虽然这些研究证明了把 LLM、KG 和检索机制结合起来的价值，但由此产生的图结构一般用于组织领域知识、支持信息检索或增强诊断推理。据我们所知，**对「自动构建可执行功能模型」这件事关注有限**——这种模型要保留诊断传播所需的层次功能依赖和逻辑关系。对基于 DML 的表示来说这一区别尤为重要，因为模型的结构直接决定诊断推理的有效性。

Beyond supporting diagnostic reasoning, executable functional models preserve engineering knowledge in a structured representation that facilitates safety assessment, maintenance planning, and engineering decision-making.

除了支持诊断推理，可执行的功能模型还把工程知识保存在一种结构化表示中，便于安全评估、维护规划和工程决策。

3. 方法总览：怎么把 LLM 的自由度收窄

The framework is designed to support the following key capabilities: (a) Extracting functional elements and their dependencies from system documentation using retrieval-augmented LLMs, allowing functional decomposition from unstructured text. (b) Layer-wise model construction for a complex system using targeted retrieval, allowing relevant portions of large-scale documentation to be selectively incorporated at each stage of the DML hierarchy. (c) Systematic evaluation of the constructed KG-DML through layer-level precision and recall metrics combined with an integrity measure reflecting structural completeness and consistency.

该框架旨在支持以下关键能力：（a）用检索增强的 LLM 从系统文档中抽取功能元素及其依赖，实现从非结构化文本进行功能分解。（b）用有针对性的检索对复杂系统做**逐层**模型构建，使大规模文档的相关部分能在

DML 层次的每个阶段被选择性地纳入。（c）通过分层的 precision 与 recall 指标，结合一个反映结构完整性与一致性的完整度度量，对构建出的 KG-DML 做系统评测。

A key concern in applying LLMs to KG construction is reliability, particularly when models are tasked with extracting arbitrary entities and relationships from large, noisy, heterogeneous documents. The framework presented here addresses this challenge by constraining the extraction process through a set of design choices that reduces ambiguity and limits the degrees of freedom available to the model. Rather than operating in an open-ended setting, the LLM is guided by a predefined schema, a structured template derived from the DML formalism, specifying node types, relationships, and hierarchical structure. In addition, the extraction task is restricted to the bounded scope of the system being modeled. Additionally, the LLM generates candidate elements and relationships, which are then verified and stored in a JSON format. A separate module transforms this data into graph database queries and builds a connected graph that can be traversed. Together, these design choices shift the role of the LLM from unconstrained generation to structured interpretation within a controlled KG construction framework.

把 LLM 用于 KG 构建时，一个关键顾虑是可靠性，特别是当模型被要求从大量、有噪声、异构的文档中抽取任意实体和关系时。本文提出的框架通过**一组降低含糊性、限制模型自由度的设计选择**来约束抽取过程，以应对这一挑战。LLM 不在开放式设定中工作，而是由一个**预定义 schema** 引导——这是一个从 DML 形式化推导出来的结构化模板，规定了节点类型、关系和层次结构。此外，抽取任务被限制在**正在建模的这个系统**这一有界范围内。再者，LLM 生成候选元素和关系，随后被验证并以 JSON 格式存储。一个独立模块把这些数据转换成图数据库查询，构建出可遍历的连通图。这些设计选择合起来，把 LLM 的角色**从无约束生成转变为受控 KG 构建框架内的结构化解释。**

4.1 模型构建的三个阶段

Model construction proceeds sequentially along the DML hierarchy, beginning with system goals and progressing through functions, subfunctions, components, and success conditions, where each success condition represents a specific performance indicator that defines whether a component fulfills its intended function.

模型构建沿 DML 层次顺序推进，从系统目标开始，依次经过功能、子功能、部件和成功条件——其中每个成功条件代表一个具体的性能指标，用来定义某个部件是否履行了它预定的功能。

At each layer, a retrieval query is constructed using the semantic definition of the current DML layer together with the parent elements identified in the previous stage. The most relevant portions of the system documentation are retrieved and supplied to the LLM, which generates candidate elements and their logical relationships in a structured format. The experiments were conducted using GPT-4o (OpenAI API), with the temperature parameter set to 0 to enforce deterministic model outputs. As validated elements are accepted, they are incorporated into a JSON representation. This master JSON representation evolves incrementally and serves as the

record of all verified elements and relationships. For each subsequent layer, the parent nodes stored in the JSON are extracted and incorporated into the next retrieval query, thereby constraining evidence selection to documentation relevant to the already-constructed portion of the hierarchy.

在每一层，检索查询由两部分拼成：**当前 DML 层的语义定义**，加上**上一阶段已识别的父元素**。系统文档中最相关的部分被检索出来并提供给 LLM，后者以结构化格式生成候选元素及其逻辑关系。实验使用 GPT-4o（OpenAI API），temperature 设为 0 以强制确定性输出。通过验证的元素被接受后并入一个 JSON 表示。这份 master JSON 增量演进，充当所有已验证元素和关系的记录。对每个后续层，从 JSON 中取出已存的父节点并并入下一次检索查询，**从而把证据选择约束在与已建好的那部分层次相关的文档上**。

阶段 1：文档处理与嵌入。 Prior to embedding and retrieval, the system documentation is preprocessed. Component identifiers and tags are normalized to a consistent format to prevent inconsistent references to the same entity. Abbreviations are expanded and defined at first occurrence. When tagged components are first introduced, identifiers are supplemented with brief descriptive context, such as functional role or location, to associate each identifier with its meaning. This context is retained during embedding so that components with similar names, but different roles are less likely to be conflated during retrieval. Where possible, pronouns and shorthand references are resolved to explicit identifiers.

在嵌入和检索之前，系统文档要经过预处理。部件标识符和标签被归一化成一致的格式，以防止对同一实体的引用不一致。缩写在首次出现时被展开并定义。带标签的部件首次引入时，标识符会补上简短的描述性上下文（如功能角色或位置），把每个标识符和它的含义关联起来。这个上下文在嵌入时保留，使名字相似但角色不同的部件在检索时不那么容易被混为一谈。在可能的地方，代词和简写引用被解析成显式标识符。

Following preprocessing, the documentation is segmented into text chunks using a paragraph-based strategy. Chunks are limited to 1,500 characters, with an overlap of 150 characters between adjacent chunks to preserve contextual continuity. Each chunk is embedded using the text-embedding-3-small model and stored in a vector database.

预处理之后，文档按段落策略切分成文本块。每块限制在 1,500 字符，相邻块之间重叠 150 字符以保持上下文连续性。每块用 text-embedding-3-small 模型做嵌入并存入向量数据库。

阶段 2：逐层构建。 Rather than attempting to infer the entire model in a single step, each layer is constructed independently while being conditioned on the results of previous layers. To manage prompt length and control the scope of generation, parent elements at each layer are processed either individually or in small batches. A batch corresponds to a fixed number of parent elements that are processed together in a single model invocation. Similarity is measured using cosine similarity between the query embedding and each document chunk embedding, and the top K chunks with the highest similarity scores are selected as supporting evidence, where K = 10 in this work. Output is restricted to a predefined JSON schema to maintain consistency and enable automated verification. Once generated, the output is checked for

structural correctness, identifier validity, and adherence to layer-specific constraints. Valid elements and links are appended to the master JSON. Outputs that fail these checks are logged, and the corresponding layer is flagged as low confidence.

与试图一步推出整个模型不同，**每一层独立构建，同时以前面各层的结果为条件**。为控制提示词长度和生成范围，每层的父元素或单独处理、或按小批处理。一个 batch 对应在单次模型调用中一起处理的固定数量的父元素。相似度用查询嵌入与各文档块嵌入之间的余弦相似度衡量，取相似度最高的 top K 个块作为支持证据，本文中 K = 10。输出被限制为预定义的 JSON schema，以保持一致性并支持自动验证。生成后，输出要检查结构正确性、标识符有效性以及对该层特有约束的遵守情况。有效的元素和连接被追加到 master JSON。未通过这些检查的输出被记入日志，对应的层被标记为低置信度。

阶段 3：图合成与导出。 As shown in Figure 5, the master JSON is passed to a query generation module that translates the structured representation into executable graph database statements. For relationships that include Boolean structure (AND/OR), the logical semantics are represented explicitly in the graph. When a dependency is governed by a gate, an intermediate gate node is created, connecting the parent element to its child nodes. If nested gates are present, the same procedure is applied recursively to preserve the multi-level logical structure without flattening the hierarchy.

master JSON 被交给一个查询生成模块，把这个结构化表示翻译成可执行的图数据库语句（本文用 Neo4j 的 Cypher）。**对包含布尔结构（AND/OR）的关系，逻辑语义在图中被显式表示**：当一个依赖由某个门控制时，会创建一个中间的 gate 节点，把父元素连到它的子节点。如果存在嵌套的门，同一过程递归应用，**以保留多层逻辑结构而不把层次扁平化。**

（Algorithm 1 给出了带嵌套布尔门的 Cypher 生成过程：先为五类节点建唯一性约束，再逐类 merge 节点，然后用一个递归的 BUILDGATE 过程处理门与子门，最后按四类连接关系——目标到功能、功能到子功能、子功能到部件、部件到成功条件——分别建边。边的类型分别是 ACHIEVED_BY、DEPENDS_ON、REQUIRES、SUCCESS_THROUGH。）

4.2 评测框架：三层指标加一个完整度分数

Before comparison, both the constructed KG-DML and the reference model are converted into a normalized representation. This step reduces differences that do not affect model meaning, such as variations in naming or identifier assignment. Element names are standardized using string normalization and semantic matching so that conceptually equivalent nodes can be aligned even when their textual labels differ slightly.

在比较之前，构建出的 KG-DML 和参考模型都被转换成归一化表示。这一步消除那些不影响模型含义的差异，比如命名或标识符分配上的变化。元素名通过字符串归一化和语义匹配被标准化，使概念上等价的节点即使文本标签略有不同也能对齐。

Each predicted node is classified into one of three categories: correctly identified, missing, or hallucinated. These classifications are used to compute precision and recall at each layer. An F2

score is also reported to place greater emphasis on recall, reflecting the fact that missing elements in a DML model are generally more detrimental than the inclusion of extra elements.

每个预测节点被归入三类之一：正确识别、缺失、幻觉。这些分类用于计算每层的 precision 和 recall。同时报告 **F2 分数以更偏重 recall**，这反映了一个事实：在 DML 模型里，**漏掉元素通常比多包含元素更有害。**

Since logical relationships between parent and child nodes are represented through AND/OR gates at each hierarchical layer, gate accuracy is evaluated for all layers. Gate correctness is evaluated separately from node and link identification, as an incorrect logical operator can alter how elements contribute to higher-level functions even when the correct nodes are present. Errors occurring at higher levels of the hierarchy tend to propagate downstream and affect larger portions of the model. As a result, node omissions, relationship errors, and gate mismatches are weighted according to the DML layer at which they occur.

由于父子节点之间的逻辑关系在每个层次上通过 AND/OR 门表示，门准确率对所有层都要评。**门的正确性与节点、连接的识别分开评测，因为逻辑运算符错了会改变元素对上层功能的贡献方式，即使正确的节点都在。**发生在层次较高处的错误往往向下游传播、影响模型更大的部分。因此，节点遗漏、关系错误和门不匹配都按其所在的 DML 层加权。

Layer-specific precision, recall, and F2 scores provide useful insight into the accuracy of individual elements. However, these metrics do not capture the cumulative effect of errors across layers, nor do they reflect whether the constructed model preserves coherent functional paths from system goals to success conditions. To address this limitation, an overall integrity score is defined.

分层的 precision、recall 和 F2 分数能反映单个元素的准确性。但这些指标**捕捉不到跨层错误的累积效应**，也不反映构建出的模型是否保留了从系统目标到成功条件的连贯功能路径。为解决这一局限，定义了一个整体完整度分数：

> $\text{IntegrityScore} = 100 \times (\Sigma\ w_l \cdot F_l \div \Sigma\ w_l) \times \exp(-E_{structural} \div (N_{nodes} + N_{links}) \div S)$

where F_l denotes the F2 score for layer l, w_l is the importance weight assigned to that layer, and E_structural represents the aggregate weighted structural penalty, computed from structural reconstruction errors including missing nodes, missing relationships, gate mismatches, and orphan nodes. The term (N_nodes + N_links) represents the size of the reference model, and S is a penalty scaling factor.

其中 F_l 是第 l 层的 F2 分数，w_l 是分配给该层的重要性权重，E_structural 是加权后的结构惩罚总和，由结构重建错误算出，包括缺失节点、缺失关系、门不匹配和孤立节点。(N_nodes + N_links) 代表参考模型的规模，S 是惩罚尺度因子。

The exponential form of the structural penalty was selected to reflect the nonlinear impact of accumulated structural reconstruction errors on functional reasoning. An exponential decay function increases sensitivity to accumulated errors while preserving moderate penalties for isolated errors. This formulation allows the metric to distinguish between minor local discrepancies and structural reconstruction errors that affect end-to-end reasoning. By normalizing the penalty with respect to the size of the reference model, the resulting integrity score remains comparable across systems of different sizes.

结构惩罚采用指数形式，是为了反映累积的结构重建错误对功能推理的**非线性**影响。指数衰减函数提高了对累积错误的敏感度，同时对孤立错误保持温和的惩罚。这一形式让该指标能区分「轻微的局部差异」和「影响端到端推理的结构重建错误」。通过按参考模型规模对惩罚做归一化，得到的完整度分数在不同规模的系统之间保持可比。

A score of 100 corresponds to a constructed KG-DML that fully matches the reference model in terms of nodes, relationships, and logical structure.

100 分对应构建出的 KG-DML 在节点、关系和逻辑结构上完全匹配参考模型。

4.3 交互：LLM 只解释意图并调工具

Interaction with the constructed KG-DML is mediated by an LLM-based agent equipped with a set of predefined tools. These tools implement graph traversal and probabilistic reasoning procedures and are invoked by the LLM agent based on the user's inferred intent. This interaction paradigm follows the general approach in which LLMs serve as a coordination layer that interprets user queries and selects appropriate model-based operations, rather than functioning as autonomous reasoning engines.

与构建出的 KG-DML 的交互由一个配备了预定义工具集的 LLM agent 中介。这些工具实现图遍历和概率推理过程，由 LLM agent 根据推断出的用户意图调用。这一交互范式遵循的思路是：**LLM 充当协调层，解读用户查询、选择合适的基于模型的操作，而不是充当自主推理引擎。**

向上传播。 Upward propagation evaluates system success and identifies impacted upper nodes by propagating success probability from the component level to higher-level functional and goal nodes. The computation begins at the component and success condition level, where each component is associated with an operational state (e.g., working, degraded, failed). The probabilities associated with component operational states may be derived from operational data sources, including sensor measurements, system logs, inspections, or other observational evidence. These values are encoded into the KG-DML as node attributes. The agent does not compute probabilities directly; instead, it selects and executes the tool which operates on the KG-DML. Since the tool accepts simultaneous component state assignments, multi-fault scenarios are handled through the same propagation mechanism as single-fault cases.

向上传播评估系统成功与否，并通过把成功概率从部件层向上传播到更高层的功能和目标节点，识别受影响的上层节点。计算从部件与成功条件层开始，每个部件关联一个运行状态（如工作、退化、失效）。与部件运行状态相关的概率可来自运行数据源，包括传感器测量、系统日志、检查或其他观测证据，这些值作为节点属性编码进 KG-DML。**agent 不直接计算概率，而是选择并执行在 KG-DML 上运行的工具。** 由于该工具接受同时指定多个部件状态，多故障情景与单故障情景走同一套传播机制。

向下传播。 When a user poses questions such as "How can this goal be achieved?", the tool generates minimal success path-sets by traversing the KG-DML downward through its dependency structure. The traversal respects the Boolean logic encoded in the model, including nested (AND/OR) sub-gates. For conjunctive dependencies (AND), success paths are constructed by combining the required child elements, whereas for disjunctive dependencies, alternative paths are enumerated independently.

当用户提出「这个目标怎样才能达成？」这类问题时，工具通过沿依赖结构向下遍历 KG-DML，生成**最小成功路径集**。遍历尊重模型中编码的布尔逻辑，包括嵌套的 AND/OR 子门。对合取依赖（AND），成功路径由所需的子元素组合构成；对析取依赖（OR），备选路径被独立枚举。

Common cause failure, however, may arise from both explicit structural dependencies and implicit coupling mechanisms, such as shared design, environment, maintenance practices, or underlying common root causes that are not explicitly represented in the model structure. Accordingly, when multiple identical or redundant components are identified within alternative success paths, the interaction layer augments the explanation by highlighting the potential for a CCF that could simultaneously affect these components. This augmentation does not modify the underlying logical structure. The treatment of CCF is partial, as it does not explicitly model the causal mechanisms or dependency structure within the KG-DML.

然而共因失效（CCF）既可能来自显式的结构依赖，也可能来自隐式的耦合机制——共用设计、环境、维护规程，或未在模型结构中显式表示的共同根因。因此，当在多条备选成功路径中发现相同或冗余的部件时，交互层会在解释中额外指出可能同时影响这些部件的共因失效风险。**这种补充不修改底层逻辑结构。对 CCF 的处理是部分的**，因为它没有在 KG-DML 内显式建模因果机制或依赖结构。

解释性查询。 Such questions are handled through a Graph-RAG method in which the LLM agent generates a Cypher query based on the predefined schema of the KG-DML. The agent is provided with the node labels, relationship types, and hierarchical constraints defined during model construction, allowing it to produce syntactically valid and consistent queries. In this mode, the LLM does not perform reasoning over the structure directly. Instead, it acts as a query generator and response formatter, while the graph database performs structural retrieval.

这类问题通过 Graph-RAG 方法处理：LLM agent 基于 KG-DML 的预定义 schema 生成 Cypher 查询。agent 被提供了模型构建期间定义的节点标签、关系类型和层次约束，使它能产出语法有效且一致的查询。**在这一模式下，LLM 不直接对结构做推理，而是充当查询生成器和响应格式化器，结构检索由图数据库完成。**

5. 案例：一座退役沸水堆的低压冷却剂注入系统

The case study uses documentation of the Low-Pressure Coolant Injection (LPCI) safety system used to protect the now decommissioned Millstone Point Unit 1 Boiling Water Reactor (BWR) in cases of loss of normal cooling. The documents used are from the Interim Reliability Evaluation Program prepared for the U.S. Nuclear Regulatory Commission. The LPCI system is intended to provide coolant to the reactor vessel following a loss-of-coolant accident (LOCA), ensuring adequate core cooling and preventing fuel damage under low-pressure conditions. The system is arranged as two redundant subsystems, designated A and B, which are interconnected by a normally locked-open crosstie line. The presence of nested logical groupings and alternative functional paths makes the LPCI system a suitable case study for evaluating automated DML construction.

案例使用低压冷却剂注入（LPCI）安全系统的文档——该系统用于在正常冷却丧失时保护现已退役的 Millstone Point 1 号沸水堆。所用文档来自为美国核管会编制的《临时可靠性评估计划》。LPCI 系统的作用是在冷却剂丧失事故（LOCA）之后向反应堆压力容器提供冷却剂，确保堆芯充分冷却、在低压条件下防止燃料损坏。系统布置为两个冗余子系统 A 和 B，由一条常态锁定开启的横连管路相互连接。**嵌套的逻辑分组和多条备选功能路径的存在，使 LPCI 系统成为评测自动化 DML 构建的合适案例。**

6. 结果：高层完美，低层微差，批越大越不稳

All quantitative results are reported across five independent executions of the model on the same system description and configuration. Minor variations arise from residual nondeterminism in the pipeline, including differences in retrieval results and LLM inference, which may propagate through the hierarchical construction process.

所有定量结果都基于对同一系统描述和配置的**五次独立执行**。微小差异来自流水线中残余的非确定性，包括检索结果和 LLM 推理的差异，它们可能在层次构建过程中传播。

At the node level, identification is perfect and consistent for the Goals, Functions, and Subfunctions layers, with precision, recall, and F2 equal to 1.0 across all runs. For Components, F2 ranges from 0.978 to 0.990, corresponding to approximately 1-3 missed nodes per run. For Success Conditions, F2 ranges from 0.980 to 0.993, corresponding to approximately 2-6 missed nodes per run.

在节点层面，Goals、Functions 和 Subfunctions 三层的识别**完美且一致**，五次运行的 precision、recall 和 F2 全部等于 1.0。Components 层 F2 在 0.978 到 0.990 之间，对应每次约漏 1-3 个节点。Success Conditions 层 F2 在 0.980 到 0.993 之间，对应每次约漏 2-6 个节点。

At the link level, higher-level relationships (Goal-Function and Function-Subfunction) are reconstructed perfectly across all runs. Variability is concentrated at the Subfunction-Component layer, where structural density is highest. At this level, recall ranges from 0.951 to

0.978, F2 from 0.961 to 0.982, and gate accuracy from 0.889 to 0.963. In contrast, the Component-Success Condition layer exhibits near-perfect performance, with F2 ranging from 0.980 to 0.993, and gate accuracy consistently equal to 1.0.

在连接层面，较高层的关系（目标-功能、功能-子功能）在所有运行中都被完美重建。变异集中在**结构密度最高的子功能-部件层**：该层 recall 在 0.951 到 0.978，F2 在 0.961 到 0.982，**门准确率在 0.889 到 0.963**。相比之下，部件-成功条件层表现接近完美，F2 在 0.980 到 0.993，门准确率始终等于 1.0。

The highest score is achieved with a batch size of 1, with performance decreasing gradually as batch size increases. This trend indicates a trade-off between structural consistency and computational efficiency.

完整度分数在 batch size = 1 时最高，随 batch size 增大逐渐下降。这一趋势表明**结构一致性与计算效率之间存在权衡**。

运行	Batch=1	Batch=5	Batch=10	Batch=15
1	92.82	86.86	91.06	89.20
2	91.14	87.37	86.60	87.48
3	89.60	88.66	86.97	86.16
4	88.21	91.06	85.95	84.32
5	90.12	90.93	89.31	84.57
均值	**90.38**	**88.98**	**87.98**	**86.35**
标准差	1.73	1.96	2.14	2.04

Increasing the penalty scale leads to higher overall scores and reduced variability, while preserving the relative ranking across configurations. This behavior indicates that the integrity metric is sensitive to penalty scaling in magnitude but remains stable in relative comparison across configurations.

增大惩罚尺度会带来更高的总分和更小的变异，同时保持各配置之间的相对排序。这说明**完整度指标在数值量级上对惩罚尺度敏感，但在配置间的相对比较上保持稳定**。（batch=5 时，惩罚尺度 0.4/0.5/0.6 对应的均值分别为 86.52 / 88.98 / 90.65，标准差 2.35 / 1.96 / 1.68。）

诊断结果举例。 When the user queries about the effect of the simultaneous failure of the heat exchanger inlet and outlet valves, the system identifies the resulting impact across the hierarchy. The disruption first affects Subfunction SF3.2, then propagates to Function F3, and ultimately impacts Goal G1 (Provide emergency low-pressure coolant injection to prevent core damage during a LOCA). The output traces the functional consequence from component-level failure to the system-level objective. For subfunctions supported by redundant identical components, the system returns alternative success paths, while the explanation layer highlights the potential for CCF due to shared design or operating conditions.

当用户询问热交换器进出口阀同时失效的影响时，系统识别出这一影响在整个层次上的传导：扰动先影响子功能 SF3.2，随后传播到功能 F3，最终影响目标 G1（在 LOCA 期间提供紧急低压冷却剂注入以防止堆芯损坏）。输出把功能后果从部件级失效一路追到系统级目标。对于由冗余同型部件支撑的子功能，系统返回多条备选成功路径，同时解释层指出因共用设计或运行条件而可能存在共因失效。

7. 讨论：省下的是几个月，但作者自己给这个数字打了折

In conventional practice, constructing a DML model for a complex engineered system often requires several months of manual effort. The framework presented in this study streamlines this process by leveraging LLM-assisted extraction and KG representation, reducing model construction time to a matter of days for complex systems while preserving logical consistency and diagnostic capability.

在常规实践中，为一个复杂工程系统构建 DML 模型往往需要**数月**的人工投入。本研究提出的框架通过 LLM 辅助抽取和 KG 表示简化了这一过程，把复杂系统的建模时间**缩短到数天**，同时保持逻辑一致性和诊断能力。

Lower-level elements are more numerous and more closely tied to specific contextual descriptions, making them more sensitive to retrieval scope and batching decisions.

低层元素数量更多、与具体的上下文描述绑定更紧，因此对检索范围和批处理决策更敏感。

Although a batch size of 1 produced the highest mean integrity score, the improvement relative to a batch size of 5 was limited. Reducing the batch size from 5 to 1 increased the number of LLM invocations approximately fivefold, from roughly 32 to 158 calls across all construction layers. However, since smaller batch sizes process fewer parent elements per call, each individual prompt is shorter and requires fewer output tokens, which partially offsets the increase in total execution time. Nevertheless, the cumulative overhead in API usage and execution time remains substantial relative to the marginal gain in structural accuracy. A batch size of 5 provided a balance between structural consistency and computational efficiency.

虽然 batch size = 1 得到了最高的平均完整度分数，但相对于 batch size = 5 的改进有限。把 batch size 从 5 降到 1，**LLM 调用次数增加约五倍**——所有构建层加起来从约 32 次涨到约 158 次。不过由于更小的批每次调用处理的父元素更少，单个提示词更短、需要的输出 token 更少，这部分抵消了总执行时间的增长。即便如此，相对于结构准确性上的边际收益，API 用量和执行时间的累积开销仍然可观。**batch size = 5 在结构一致性与计算效率之间提供了平衡。**

It should be noted that this analysis focuses solely on computational cost and model performance. The potential reduction in human modeling effort enabled by automated DML construction is not explicitly quantified here.

需要指出，这一分析只关注计算成本和模型性能。自动化 DML 构建可能带来的人工建模投入的减少，在此并未被显式量化。

Although the framework accelerates DML construction, it is intended to support rather than replace expert modeling practice. The final review and validation of the generated model remain the responsibility of domain experts, particularly for detailed lower-level elements. The high structural accuracy and low variability observed across runs can be attributed in part to the constrained design of the extraction process. By limiting the degrees of freedom of the LLM through schema grounding, bounded scope, and structured generation, the framework reduces ambiguity and mitigates common failure modes associated with unconstrained KG construction.

虽然该框架加速了 DML 构建，它的意图是**支持而非替代**专家建模实践。生成模型的最终审核与验证仍是领域专家的责任，尤其是低层的细节元素。跨运行观察到的高结构准确性和低变异性，部分可归因于抽取过程的**受约束设计**：通过 schema 锚定、有界范围和结构化生成来限制 LLM 的自由度，框架降低了含糊性，缓解了与无约束 KG 构建相关的常见失效模式。

7.1 局限：五条，作者自己列的

First, the evaluation is conducted on a single case study system. In particular, model behavior under increased documentation volume distributed across multiple documents, higher component density, or more nested logical structures may introduce additional variability that is not fully reflected in the present results.

第一，评测只在单个案例系统上进行。特别是，在文档量更大且分布在多个文档中、部件密度更高、或逻辑结构嵌套更深的情况下，模型行为可能引入本结果未充分反映的额外变异。

Second, the evaluation framework emphasizes structural consistency at the node, link, and logical gate levels. Structural similarity does not necessarily guarantee identical logical behavior during upward or downward propagation. A comparison based on minimal cut sets or success path sets would provide a stronger test of behavioral equivalence. However, even complete cut set and path set equivalence would not constitute full functional validation of a DML-based model, as these metrics operate at the structural component level and do not capture the hierarchical decomposition of objectives into functions and subfunctions or how failures propagate through that functional hierarchy. Expert validation of the functional layer would therefore remain a necessary complement.

第二，评测框架强调节点、连接和逻辑门层面的结构一致性。**结构相似并不必然保证向上或向下传播时的逻辑行为相同。** 基于最小割集或成功路径集的比较会是对行为等价更强的检验。然而，**即便割集和路径集完全等价，也不构成对 DML 模型的完整功能验证**——因为这些指标运行在结构部件层面，捕捉不到「目标向功能与子功能的层次分解」以及「失效如何在这一功能层次中传播」。因此对功能层的专家验证仍是必要的补充。

A no-retrieval baseline was not evaluated, as full-context extraction is not feasible for the system documentation used in this study: its volume substantially exceeds practical context-window and generation limits, making retrieval a structural requirement of the framework rather

than a configurable design choice to be benchmarked against an alternative. Future work should include systematic ablation studies and baseline comparisons.

未评测「无检索」基线，因为本研究所用的系统文档做不到全上下文抽取：其体量大幅超出实际的上下文窗口与生成上限，**这使检索成为框架的结构性必需，而不是一个可以拿来与替代方案做基准对照的可配置设计选项**。未来工作应包括系统性的消融研究和基线比较。

Since model construction is conditioned on embedding-based similarity and top-k evidence retrieval, variations in chunk segmentation, retrieval ranking, or similarity thresholds may alter the contextual evidence supplied to the language model. In addition, the framework was evaluated using a single large language model configuration.

由于模型构建以基于嵌入的相似度和 top-k 证据检索为条件，分块切分、检索排序或相似度阈值的变化都可能改变提供给语言模型的上下文证据。此外，框架只在单一大语言模型配置下被评测。

Although sensitivity to the penalty scale parameter was examined, the selection of layer weights and the structure of the penalty term reflect modeling decisions. Alternative weighting schemes or penalty formulations could produce different numerical scores even when underlying structural discrepancies are comparable.

虽然考察了对惩罚尺度参数的敏感性，但**层权重的选择和惩罚项的结构本身是建模决策**。不同的加权方案或惩罚形式，即使底层结构差异相当，也可能得出不同的数值分数。

While the framework appears to reduce DML model construction time from several months of expert effort to a few days in the present case study, this estimate is based on limited practical experience rather than a controlled comparison.

虽然在本案例中，该框架看起来把 DML 建模时间从数月的专家投入缩短到数天，**这一估计基于有限的实践经验，而非受控比较**。

7.2 未来工作

In practice, system information is distributed across design specifications, operating procedures, maintenance manuals, and other technical documentation. Extending the framework to support RAG across distributed document sets would enable coordinated evidence selection from multiple sources. Such extensions would require structured cross-document retrieval, version reconciliation, and mechanisms for resolving conflicting or overlapping descriptions.

在实践中，系统信息分散在设计规格、操作规程、维护手册和其他技术文档里。把框架扩展到支持跨分布式文档集的 RAG，能实现从多个来源协同选择证据。这类扩展需要**结构化的跨文档检索、版本调和，以及解决冲突或重叠描述的机制**。

Another area for future work is modular construction across subsystems. Instead of building a single KG-DML representation at once, one could develop individual subsystem models and later

integrate them into a comprehensive model of the entire system.

另一个未来方向是跨子系统的模块化构建：不是一次性建一整个 KG-DML，而是先分别建各子系统模型，之后再整合成完整系统的综合模型。

Future work should investigate the explicit representation and quantification of CCF within the KG-DML framework. Quantitative CCF representations could be incorporated using established approaches such as explicit beta-factor modelling for multiple CCF groups, as well as alpha-factor models.

未来工作应研究在 KG-DML 框架内对共因失效的显式表示与量化。可以采用已有的方法纳入定量 CCF 表示，如针对多个 CCF 组的显式 beta 因子建模，以及 alpha 因子模型。

Enhancements to the retrieval strategy, such as adaptive chunking, hybrid keyword and embedding retrieval, or domain-tuned embedding models, could reduce sensitivity in densely connected layers such as Subfunction-to-Component relationships. Multi-model pipelines, in which one model performs identifier extraction and normalization, and another model focuses on hierarchical decomposition and logical structuring, may allow more direct ingestion of heterogeneous documentation formats with less manual normalization.

检索策略的改进——自适应分块、关键词与嵌入的混合检索、或领域微调的嵌入模型——可以降低在密集连接层（如子功能到部件的关系）上的敏感性。**多模型流水线**（一个模型负责标识符抽取与归一化，另一个模型专注层次分解与逻辑结构化）可能允许更直接地摄取异构文档格式，减少人工归一化。

Finally, engineering system documentation is often proprietary, sensitive, or subject to regulatory restrictions. In such settings, transmitting system descriptions to external APIs may not be feasible. Future work should therefore investigate locally hosted language model deployments that allow confidential documentation to remain within secure organizational boundaries.

最后，工程系统文档往往是专有的、敏感的，或受法规限制。在这类场景下，把系统描述传给外部 API 可能不可行。因此未来工作应研究**本地部署的语言模型**，使机密文档留在组织的安全边界之内。

8. 结论

By combining layer-by-layer retrieval with structured generation and graph synthesis, the approach converts unstructured technical text into a functional hierarchy that can be stored, queried, and analyzed computationally. The objective was not to replace expert modeling, but to reduce the manual effort required to extract and organize system structure from documentation.

通过把逐层检索与结构化生成、图合成结合起来，该方法把非结构化的技术文本转换成一个可存储、可查询、可计算分析的功能层次。**目标不是替代专家建模，而是减少从文档中抽取和组织系统结构所需的人工投入。**

Higher-level goals and functions were identified without variation, while minor differences appeared at the Component and Subfunction-Component layers, where structural density and logical combinations are greater.

高层的目标和功能被无差异地识别出来，而在结构密度和逻辑组合更大的部件层与子功能-部件层出现了微小差异。

In this architecture, the language model interprets user queries and invokes predefined graph-based tools, while probabilistic and logical reasoning remains governed by the encoded model structure. This separation helps maintain traceability and limits unsupported reasoning during interaction.

在这一架构中，语言模型解读用户查询并调用预定义的基于图的工具，而**概率与逻辑推理仍由编码在模型结构中的东西支配**。这种分离有助于保持可追溯性，并限制交互过程中无依据的推理。